

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

STYLE CART: Smart Fashion E-commerce Platform

submitted in partial fulfilment of the requirement of the requirement for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Parag	2210990635
Pratham Tandon	2210990676
Priyanshu	2210990685
Rahul	2210990702

Supervised By:

Dr. Gurpreet Singh
Assistant Professor
CUIET



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S. No.	Title	Page No.
	Declaration	3
	Acknowledgement	4
	Abstract	5
1.	Introduction	6-7
2.	Methodology	8
3.	Tools and Technologies	9
4.	Implementation	10-11
5.	Results	12-13
6.	Conclusion and Future Scope	14
	Reference	15

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “Style Cart: Smart Fashion E-commerce Platform” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr. Gurpreet Singh. The matter presented in this project report has not been submitted in any other university/institute for award of any degree.

Place: Rajpura
Date: 28-04-2026

Parag	2210990635
Pratham Tandon	2210990676
Priyanshu	2210990685
Rahul	2210990702

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Gurpreet Singh

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, INDIA

ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to all those who have contributed to the successful completion of our project, *StyleCart: Smart Fashion E-commerce Platform*.

First and foremost, we are deeply indebted to our project supervisor for his invaluable guidance, continuous support, and insightful suggestions throughout the development of this project. His expertise and encouragement played a crucial role in shaping the direction and quality of the work.

We would also like to extend our gratitude to the faculty members of our department for providing us with the necessary academic knowledge and technical foundation required to undertake this project. Their teachings have been instrumental in helping us understand key concepts related to software development and system design.

We are thankful to our institution for providing the infrastructure, resources, and a conducive learning environment that enabled us to carry out this project efficiently.

We would like to acknowledge our peers and friends for their cooperation, constructive discussions, and moral support during the development phase. Their feedback and encouragement helped us stay motivated and overcome various challenges.

Finally, we express our deepest gratitude to our families for their unwavering support, patience, and encouragement throughout this journey. Their belief in us had been a constant source of strength.

We take full responsibility for any shortcomings in this work and view them as opportunities for further learning and improvement.

ABSTRACT

StyleCart – Smart Fashion E-commerce Platform is an original computer software application proposed to be developed as an academic project. The software is intended to provide a web-based platform for managing fashion-related products such as clothing, footwear, and accessories in an organized and user-friendly manner.

The proposed system is designed to support functionalities including user registration and authentication, product categorization, browsing and filtering of products, shopping cart management, order placement, and basic order tracking.

An administrative module is planned to facilitate product listing, inventory management, and order monitoring to support efficient backend operations. The software is being developed using the Django framework for backend processing and database management, along with HTML, CSS, JavaScript, and Django templates for the frontend interface.

The project follows a modular and secure software design approach. The source code, system architecture, and interface design constitute original academic work. The software is currently unpublished and has not been commercially distributed.

The work is intended solely for academic purposes and is suitable for copyright protection as computer software.

1. Introduction

1.1. The Evolution and Significance of E-Commerce Systems

An E-Commerce system using Django with a Graphical User Interface (GUI) represents a sophisticated, dynamic web application engineered to bridge the gap between digital storefronts and global consumers. In the contemporary digital era, the landscape of retail has undergone a seismic shift; the transition from brick-and-mortar establishments to expansive online marketplaces is no longer a luxury but a necessity for business survival. This rapid proliferation is fuelled by the ubiquity of high-speed internet accessibility, the maturation of secure digital payment ecosystems, and a fundamental pivot in consumer psychology toward "anytime, anywhere" shopping. For businesses, these platforms remove geographical barriers, allowing a small local enterprise to achieve a global market reach. For customers, they provide an unparalleled level of convenience, enabling them to compare prices, read reviews, and execute transactions through efficient, automated processes that save both time and resources.

1.2. Leveraging the Django Framework for Robust Development

The primary objective of this project is to architect a high-performance, full-fledged E-Commerce web application by harnessing the power of the Django framework. Django was specifically selected for this endeavour due to its reputation as the "web framework for perfectionists with deadlines." It offers an out-of-the-box toolkit that emphasizes scalability, top-tier security features (such as protection against SQL injection and cross-site scripting), and rapid development cycles. At the heart of this system lies Django's Model-View-Template (MVT) architecture. This design pattern is crucial for maintaining a clean separation of concerns: the Model handles the data structure and database interactions, the View manages the business logic and processes user requests, and the Template governs the presentation layer. This organized structure ensures that the codebase remains maintainable, modular, and easily extensible as the application grows in complexity.

1.3. Multi-User Functionality and Role-Based Access

To ensure a comprehensive shopping ecosystem, this project is meticulously designed to cater to two distinct user archetypes: Customers and Administrators. For the customer, the application provides an end-to-end journey—beginning with intuitive product discovery and advanced filtering, moving through a persistent shopping cart system, and culminating in a secure checkout and payment gateway. Customers also benefit from personalized accounts where they can track order statuses and review their purchase history. Conversely, the administrative side of the platform serves as the command centre. Through a secure dashboard, administrators possess the authority to manage the product catalogue (adding, updating, or deleting items), oversee user accounts, monitor inventory levels, and

process incoming orders. This dual-layered approach ensures that the platform is as functional for the service provider as it is for the end consumer.

1.4. Front-End Design and Responsive User Experience

A critical component of any modern web application is its accessibility across diverse hardware, and this project prioritizes a seamless Graphical User Interface (GUI). By integrating HTML5, CSS3, and Bootstrap, the front end is built to be "mobile-first" and fully responsive. This means the interface dynamically adjusts its layout whether it is being accessed from a desktop monitor, a tablet, or a smartphone, ensuring that the user experience remains consistent and professional. Bootstrap's grid system and pre-styled components allow for a polished, modern aesthetic that builds user trust—a vital factor in e-commerce. The integration of these front-end technologies with the Django backend ensures that the application is not only powerful and secure but also visually engaging and easy to navigate.

1.5. Project Scope and Objectives

This introductory chapter establishes the foundational framework for the entire project, outlining the specific objectives and the socio-economic drivers behind the need for a custom e-commerce solution. By identifying the limitations of traditional retail and the advantages of the Django ecosystem, we set a clear path for the technical implementation described in subsequent chapters. This project aims to go beyond basic functionality, seeking to address the nuanced requirements of the modern online shopping experience—such as data integrity, user privacy, and system performance. By clearly defining the problem domain and the scope of the features provided, we create a roadmap for developing a scalable solution capable of meeting the demands of today's digital marketplace.

2. Methodology

2.1. The Agile Development Approach

The methodology adopted for this project is the Agile Development Life Cycle, which emphasizes iterative progress, flexibility, and continuous refinement. Unlike the traditional Waterfall model, Agile allowed for the simultaneous development of various modules—such as the product catalogue and user authentication—enabling early detection of bugs and architectural flaws. By breaking the project into "sprints," specific features were built, tested, and integrated in cycles. This approach ensured that the Graphical User Interface (GUI) could be refined based on immediate visual feedback, ensuring the final product aligned perfectly with user expectations and modern web standards.

2.2. Requirement Gathering and Feasibility Study

Before any coding commenced, an extensive feasibility study was conducted to analyse the technical, operational, and economic viability of a Django-based e-commerce solution. This phase involved gathering functional requirements—such as secure payment processing and inventory management—as well as non-functional requirements like system latency and data encryption standards. Stakeholder interviews and market analysis of existing platforms like Shopify and WooCommerce helped in identifying the core features needed to remain competitive. This foundational research ensured that the project scope was well-defined, preventing "feature creep" and ensuring a focused development path.

2.3. System Modelling and Prototyping

To bridge the gap between requirements and code, detailed system models were created using Unified Modelling Language (UML) diagrams. This included **Use Case Diagrams** to map user interactions, **Entity-Relationship Diagrams (ERD)** to structure the database, and **Data Flow Diagrams (DFD)** to visualize how information moves from the product gallery to the final invoice. Low-fidelity wireframes were then developed to map the user journey, ensuring the GUI was intuitive. These blueprints acted as the architectural "north star," guiding the backend logic and frontend design to ensure they remained synchronized throughout the implementation phase.

3. Tools and Technologies

3.1. Backend: Django Framework

The backbone of this application is Django 4.x, a high-level Python web framework. Django was chosen for its "batteries-included" philosophy, which provides built-in tools for handling user authentication, site maps, and RSS feeds right out of the box. Its powerful Object-Relational Mapping (ORM) allows for complex database queries to be written in Python code rather than raw SQL, significantly reducing development time and minimizing the risk of injection attacks. Furthermore, Django's automated administrative interface provided a ready-made command centre for managing products and orders, allowing development efforts to focus on the unique customer-facing features.

3.2. Frontend and UI Frameworks

To create a modern, responsive GUI, a combination of **HTML5**, **CSS3**, and **Bootstrap 5** was employed. Bootstrap was instrumental in ensuring the website is "device-agnostic," meaning the layout fluidly adjusts for smartphones, tablets, and desktops. For client-side interactivity—such as dynamic cart updates without page reloads—**JavaScript (ES6+)** and **AJAX** were utilized. This technology stack ensures that the presentation layer is not only visually professional but also performance-optimized, providing the snappy, interactive experience that modern consumers expect from top-tier e-commerce platforms.

3.3. Database and Version Control Tools

Data persistence is managed via SQLite during the development phase, with architectural support for PostgreSQL in production environments to handle high-concurrency transactions. For version control and collaborative coding, Git and GitHub were used extensively. This allowed for meticulous tracking of code changes, branch management for testing new features, and a secure backup of the entire codebase. Other essential tools included Pip for Python package management and Postman for testing API endpoints and payment gateway integrations, ensuring every technical component was rigorously verified.

4. Implementation

4.1. Backend Architecture and Django App Modularization

The system is built on Django 4.x, utilizing a modular architecture to ensure code maintainability. The project is partitioned into several independent "apps," each dedicated to a specific functional domain. The accounts app manages the user lifecycle, including registration and secure authentication. The products app serves as the digital storefront engine, handling the product catalogue and inventory logic. The cart app facilitates temporary item storage via session management, while the orders app captures finalized selections, saving order history and transaction details to the database for administrative review.

4.2. Frontend Development and Template Inheritance

The presentation layer is developed using Django Templates integrated with Bootstrap 5 and JavaScript for responsiveness. To minimize code redundancy, the project utilizes Template Inheritance. A primary base.html file defines the global structure, including the navigation bar and footer. Specific pages, such as index.html for the homepage and product_detail.html, extend this base file. This ensures a consistent user interface across all devices and allows for global design updates to be implemented from a single source file, significantly streamlining the frontend development process.

4.3. User Authentication and Role-Based Access

Security is managed through Django's built-in User model, providing robust password hashing and session security. A Custom User Profile model is linked to the base user to store additional data like contact information and addresses. The system enforces Role-Based Access Control (RBAC) to distinguish between two user types:

- Customers: Restricted to the frontend interface for browsing and placing orders.
- Administrators: Granted access to the Django Admin Panel, a secure backend command centre where they can manage users, modify the product catalogue, and monitor incoming order requests.

4.4. Product Management and Catalogue Logic

The implementation of the product catalogue relies on Django Model Forms, which automate the interaction between the database and the administrative interface. This allows for seamless management of product metadata, including high-resolution image uploads, pricing, and stock status. On the frontend, the catalogue is optimized with search functionality and category filtering to enhance user navigation. The use

of "Slug" fields ensures that every product has a unique, SEO-friendly URL, improving the platform's visibility and making the browsing experience more intuitive for the end-user.

4.5. Order Processing and Management Workflow

The final stage of implementation involves the logic for transitioning from a shopping cart to a finalized order. Since the system follows a Cash on Delivery (COD) model, the workflow is designed to bridge the gap between digital selection and physical fulfilment. Once a user confirms their cart, the system executes a backend view that clears the session-based cart and generates a new entry in the Order table. Each order is assigned a unique tracking ID and a "Pending" status. On the administrative side, the Django Admin dashboard provides a streamlined interface for the shop owner to view these orders, verify customer contact details, and update the status to "Dispatched" or "Delivered" once the physical transaction is complete. This ensures a transparent and reliable order lifecycle without the need for complex payment gateway integration.

5. Results

5.1. Mastery of the Django MVT Architecture

One of the most significant outcomes of this project was a deep, practical understanding of the Model-View-Template (MVT) architectural pattern. By decoupling the data logic (Models) from the user interface (Templates) and the processing logic (Views), the development process became highly organized and manageable. This separation allowed for rapid debugging and feature implementation, such as the session-based cart system, without disrupting the overall site structure. Learning to leverage Django's built-in tools, such as the automated Admin Panel and Object-Relational Mapping (ORM), significantly reduced the time spent on boilerplate code and highlighted why Django is a premier choice for scalable web applications.

5.2. Responsive Design and Frontend Integration

A major technical achievement was the successful integration of a dynamic backend with a fully responsive frontend using Bootstrap 5. The project provided hands-on experience in "mobile-first" design, ensuring that the e-commerce store was visually appealing and functional across various screen sizes, from smartphones to desktop monitors. Utilizing Template Inheritance allowed for a consistent UI/UX across the site, while the addition of JavaScript for interactive elements—like the dynamic updating of cart totals—taught the importance of client-side responsiveness. This synergy between Python logic and modern CSS frameworks resulted in a professional-grade interface that rivals commercial platforms.

5.3. Database Management and Data Integrity

Developing the project's database schema served as an essential learning experience in relational database management. Designing the relationships between Users, Products, and Orders using Foreign Keys ensured strict data integrity, preventing errors such as orders being placed for non-existent products. Utilizing Django Migrations to manage schema changes provided a safe way to evolve the database as the project scope expanded. This outcome underscored the importance of a well-planned database structure in ensuring that every transaction is recorded accurately and that the inventory remains synchronized with the customer-facing storefront.

5.4. Security Best Practices and System Robustness

The project provided critical insights into web security and the importance of protecting user data. By implementing Django's built-in security features, such as CSRF (Cross-Site Request Forgery) tokens for order submissions and secure

password hashing for user registration, the system was made resilient against common web vulnerabilities. Learning to manage user roles and permissions through the Admin interface reinforced the concept of "least privilege," ensuring that only authorized personnel could modify the product catalog or access sensitive order data. These security implementations resulted in a robust platform that prioritizes user safety and maintains a high level of operational trust.

6. Conclusion and Future Scope

The development of the E-Commerce System using Django with GUI successfully meets the project's objectives of providing a feature-rich and user-friendly platform for online shopping. The system allows users to browse and purchase products, while administrators manage listings, monitor transactions, and analyze orders.

Throughout the project, Django proved to be an effective framework, offering modularity, built-in security, and rapid development capabilities. The system integrates multiple components—user authentication, product management, cart functionality, order processing, and payment gateway integration—into a cohesive platform. The graphical user interface ensures an intuitive experience for both customers and administrators.

The system is scalable and secure, capable of handling increasing user loads and data volumes. Testing and evaluation confirmed its performance under different conditions, highlighting reliability and efficiency.

However, technology is ever-evolving, and the platform can benefit from future enhancements. Suggested features include:

- Integration of AI for personalized product recommendations.
- Real-time order tracking with shipment APIs.
- Loyalty programs and discount codes for customer retention.
- A native mobile application for improved accessibility.
- Enhanced analytics dashboards for admin users.

Moreover, SEO optimization, multilingual support, and chatbots for customer support can further enhance usability and market reach.

In conclusion, this project serves as a foundational model for scalable, modern e-commerce solutions. It not only fulfils the technical objectives but also showcases the capabilities of Django in real-world applications. With continuous updates and integration of advanced features, the system holds strong potential for real-world deployment in commercial settings.

References

- Django Software Foundation (2026). *Django Documentation: The Web Framework for Perfectionists*. This is the definitive guide for understanding the MVT architecture, ORM queries, and session-based cart management.
- Website: <https://docs.djangoproject.com/en/stable/>
- The Python Software Foundation (2026). *Python Language Reference and Standard Library*. Essential for implementing the core logic and data structures used within the Django apps.
- Website: <https://docs.python.org/3/>
- Bootstrap Core Team (2026). *Bootstrap v5.3 Documentation*. Provided the CSS framework and grid system logic necessary to create the responsive, mobile-first Graphical User Interface (GUI).
- Website: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>